TrustEdge Phase 2 guide to the project and Raspberry Pi setup

Prepared on 1 November 2026 for the project owner and the friend continuing the hardware work.

1 What is ready today

TrustEdge has a working software foundation and a local dashboard. It is prepared to move from the computer demonstration to a Raspberry Pi 4. It has not yet been tested with a physical Pi or real connected devices, so it still needs installation and practical checks before it can protect a real network.

The main idea is simple: devices send their traffic through the Pi. The Pi learns each device's usual behavior, watches for changes, and decides how much access that device should have. It remembers its decisions on its own storage.

All learning, decisions, saved records, and dashboard pages stay on the machine running TrustEdge. There is no cloud monitoring service and no automatic download of replacement models. Ordinary Internet activity from an allowed device can still reach the Internet. If an outside service is chosen to look up website names, those ordinary name requests can also leave the Pi.

The Pi will run the project itself. It is not an accessory that must remain connected to your Mac to make decisions. Your computer can open the Pi's dashboard to manage it, while the Pi continues doing the work.

2 What you see in the demonstration

The Studio laptop, Living room camera, Office printer, Hallway thermostat, and Guest device are examples created by the demonstration program. They are not discovered devices in your home. They are not separate computers or Docker containers.

Each example is a saved record containing a name, a pretend device address, a device category, a trust score, and an access level. The demonstration creates example traffic descriptions and passes them through the actual learning and decision code. It does not send those example packets onto your network or change your computer's firewall.

The example records are remembered between launches. A device that was quarantined yesterday can still be quarantined today. Restarting the dashboard does not erase that saved decision. Earlier demonstrations used an immediate-quarantine rule; saved 10 percent scores from those runs can remain even though the rule has since changed.

The demonstration deliberately includes different device states so you can explore the dashboard. Its Guest device is given repeated scan activity to show a confirmed quarantine. The other examples may change when you try the simulation buttons or management actions.

Open the demonstration now

On your Mac, double-click `Dashboard.command` in the project folder. It opens the browser and signs you in automatically. Keep the terminal window it opens running. Press Control and C together in that window when you want to stop.

You can also run this from a terminal opened in the project folder:

```
.venv/bin/python -m trustedge.dashboard
```

The project's Python environment must already be installed. That is already how this workspace was prepared. The launcher normally remembers its examples in `dashboard-local.sqlite`. It is a demonstration launcher; it does not switch itself into real-device mode when a Pi is connected.

3 What has been implemented

Part of the project	What it does now
Device list	Lets an operator add a device, give it a name and category, and list the outside addresses it is expected to contact. Real devices are added deliberately rather than automatically discovered.
Real traffic collection	Contains the code to watch traffic passing through the Pi's chosen device-side connection. This is separate from the demonstration.
Behavior measurements	Turns traffic descriptions into the same ordered set of 14 measurements used by Phase 1. It does not keep message contents or packet payloads.
Learning and detection	Includes the two-part Phase 1 learning approach, its normal-behavior comparison, its checks for obvious attack patterns, and local updates as more suitable traffic is observed.
Trust decisions	Maintains a separate score for every device, remembers repeated suspicious behavior, and waits for confirmation before changing access.
Access restrictions	Contains real Pi firewall control. It checks whether a requested change succeeded instead of simply assuming that the device was restricted.
Saved state	Keeps device details, scores, learning progress, learned models, repeated-warning counts, recent traffic measurements, and activity records in a local database.
Restart handling	Reloads saved state and reapplies the intended access rules when the service starts again. Practical Pi restart and power-loss testing are still required.
Device management	Supports changing names and expected destinations, restarting learning, removing devices, and deliberately releasing quarantine with a recorded reason.
Dashboard	Provides seven local pages for overview, devices, traffic and detections, trust, the animated network map, activity history, and Pi health.
Hardware setup preparation	Generates editable setup files for the two network connections, local address assignment, website-name lookup, traffic routing, and starting the project when the Pi boots.

Part of the project	What it does now
Setup checks	Can check the Pi's local settings and report missing tools, wrong connections, conflicting address ranges, and whether traffic forwarding is enabled.

The dashboard uses the Phase 1 dark navy and purple style, with green, amber, and red for device access levels. Its animated map uses the configured Pi connections and the last confirmed device state. Moving dots illustrate possible policy paths; they are not a recording of individual packets.

4 How the learning works

A learned model is a saved description of a device's normal traffic pattern. The starting point is each device's normal behavior. TrustEdge needs a period of traffic that you know is legitimate. Passing an obvious-attack check does not prove that every learning sample is safe.

1. Your friend adds the real device to the project and confirms that it has a stable address.
2. The device is used normally while the Pi observes it.
3. The project collects 50 suitable observation periods. Each period is about one minute and needs at least five packets, meaning small pieces of network traffic, to count.
4. The first learning model is trained from those observations. It starts comparing later behavior with the learned pattern.
5. After 15 suitable normal periods during detection, the second model learns to help reduce false alarms. Later local updates use suitable recent observations.

Fifty observation periods take at least about 50 minutes with enough traffic in every period. Quiet devices can take longer. The second model needs additional suitable traffic after the first model is trained. These are not guaranteed completion times.

The model does not know a device is safe just because it is called a camera or printer. Its name is for you; its observed behavior provides the evidence.

What is the same as Phase 1

The project carries over the Phase 1 measurement order, preparation of those measurements, learning-model settings, comparison calculations, and second-model false-alarm checks. Automated tests compare those parts with the Phase 1 code.

The original trained model files were not present in the GitHub repository. Therefore, the code can train new models from real device behavior, but it cannot recover your old trained results from the source code alone. If separately saved Phase 1 model files are available, your friend can import trusted compatible copies using the instructions in `docs/PHASE1_ML.md`.

The behavior that chooses access levels has intentional changes. In particular, a scan no longer forces an immediate 10 percent score and quarantine. The Phase 1 GitHub repository remains unchanged and has never been pushed to.

5 What the access levels mean

Access level	Meaning in this version
TRUSTED	Broad outgoing access through the Pi, with replies allowed. This still depends on the rest of the network working.
CAUTIOUS	Limited web access to explicitly listed outside addresses, plus the configured website-name lookup service. It is not unrestricted Internet access.
QUARANTINE	Ordinary outgoing traffic is blocked. The configured name lookup service and local address renewal remain available.

New devices begin CAUTIOUS while learning. A learning device can show a 75 percent score and still be CAUTIOUS; it has not completed the evidence needed for broader access.

Repeated suspicious behavior lowers trust gradually. A downgrade needs three consecutive qualifying suspicious observation periods. Each downgrade moves one level, from TRUSTED to CAUTIOUS or from CAUTIOUS to QUARANTINE. The score must also support that restriction. One scan from an otherwise healthy or newly enrolled device does not automatically quarantine it.

A recovery decision needs ten qualifying normal observation periods. Once a device is quarantined, the project does not automatically release it just because its now-restricted traffic looks quiet. An operator must review it and use **Release to cautious**, giving a reason. That action sets trust to 55 percent and grants only limited access.

In live mode these observation periods are about one minute. In simulation, pressing a button processes one example period immediately; it does not wait a real minute.

Why there are two policy columns

Desired policy is the access level the project has requested. **Confirmed policy** is the last access level whose application was reported successful. If they differ, the request is still pending or failed. Do not assume the stricter restriction is active until it is confirmed.

In the demonstration, confirmation means the simulation accepted the decision. It does not mean a real firewall was changed. On the Pi, confirmation comes from the local firewall command, and real traffic still needs to be tested.

Three actions that do different jobs

- **Release to cautious** gives a reviewed quarantined device limited access again and records your reason.
- **Rebaseline** starts learning again after a legitimate behavior change, such as a device software upgrade. It clears the old learning state but preserves trust and quarantine. It does not release a quarantined device.
- **Remove device** removes its saved device information, model, and traffic history after rule cleanup succeeds. Its activity records remain for review. An unregistered device is blocked, so removal is not a way to restore access.

6 What is still unfinished or unverified

The latest recorded full check passed 52 automated tests. Two tests that require Linux were skipped on the Mac. JavaScript checks and the check that dashboard files are included in the installable package also passed. These results verify useful software behavior; they do not prove the physical Pi setup works.

Your friend still needs to verify:

- Actual Pi installation, cable connections, traffic passing through the Pi, and reliable website-name lookup and device address renewal.
- Real restrictions for every access level, including an already-open connection when its device becomes restricted.
- Real device measurements, learning quality, false alarms, and behavior under heavier traffic.
- Service restart, an actual Pi reboot, and practical recovery after power loss.
- Browser appearance, buttons, keyboard use, charts, and small-screen layout. Browser visual testing was unavailable during development.
- Pi memory use, processor use, storage activity, temperature, and forwarding speed.

Other present limits are important:

- Devices must be added manually. The animated map is not an automatic search of your network.
- The current limited-access policy permits web traffic on ports 80 and 443 to listed addresses. Devices needing other kinds of traffic, such as some camera streams or smart-device messaging, need additional access-rule work before their normal operation is supported. Do not assume every type of IoT device works unchanged.
- The current version supports the usual dotted device addresses, such as `192.168.50.2`. The newer IPv6 address type is blocked on the protected path until it has full support.
- Stable addresses help identify devices, but this version does not prove the identity of a device or stop a device impersonating another address.
- Devices talking directly to each other through the same switch may not pass through the Pi. Extra switch or wireless isolation settings are needed if that traffic must also be controlled.
- There is one administrative access key, not separate user accounts with different permission levels.

7 The equipment and what each item does

Your planned equipment is a Raspberry Pi 4, a router, three Ethernet cables, and a USB-to-Ethernet adapter. A working Pi power supply and a storage card containing the operating system are also needed. A screen and keyboard or a previously arranged remote connection will help with the first setup.

The Pi’s built-in Ethernet socket connects toward the existing router. The USB Ethernet adapter provides a second socket for the protected test device. The Pi must sit between the test device and the router so it can observe and control that device’s traffic.

```
Existing router
      |
      | Ethernet cable 1
      |
Pi built-in Ethernet socket
Raspberry Pi 4 running TrustEdge
Pi USB Ethernet adapter
      |
      | Ethernet cable 2
      |
One wired test laptop or other test device
```

The third Ethernet cable is spare for this initial arrangement. More than one wired test device needs a switch connected behind the Pi's USB adapter. A phone or wireless-only device needs a suitable wireless access point behind the Pi, or a later wireless setup. This project does not currently set up the Pi as a wireless access point.

A test device connected to the existing router's other sockets or Wi-Fi can go around the Pi. For the first test, turn off the test laptop's Wi-Fi so its normal connection really goes through the cable to the Pi. Keep your management computer on the router side, or use the Pi's own screen and keyboard.

8 How your friend moves the project onto the Pi

The steps below explain the work in everyday language. Your friend should use `docs/PI_DEPLOYMENT.md` for the exact system files and commands. Setting up a Pi to sit between devices and a router requires those installation steps; attaching cables alone does not activate the project.

Step 1 Prepare the Pi

Install a compatible 64-bit Raspberry Pi operating system. The reference setup uses a Debian Bookworm-compatible environment with Python 3.11. Your friend should check the actual operating-system and software versions before installation because the learning-model version is pinned by this project.

First confirm that the Pi can start normally and reach the existing router. Keep a way to control the Pi that does not depend on the protected test device's Internet access.

Step 2 Copy this Phase 2 project

Copy the Phase 2 source folder to the Pi. This is the workspace that contains `trustedge`, `deploy`, `docs`, and `pyproject.toml`. Copying only the original Phase 1 GitHub repository will not provide these Phase 2 changes.

The documented Pi location is `/opt/trustedge`. Recreate the project's Python environment on the Pi and install the dependencies there. Do not copy the Mac's `.venv` folder and expect it to run on the Pi. Do not use the demonstration database as the real-device database, and keep administrative secrets out of shared source copies.

Downloading software during installation is separate from runtime operation. Once its dependencies are installed, TrustEdge does not need a cloud monitoring account or a cloud model service to work.

Step 3 Identify the two connections

Check which connection is the Pi's built-in socket and which belongs to the USB adapter. The examples call them `eth0` and `eth1`, but a USB adapter may have a longer name. The settings must use the Pi's actual names.

The built-in socket is the router side. The USB adapter is the protected-device side for this planned arrangement. Choosing the wrong names can block the wrong connection or leave the test device outside the intended protection.

Step 4 Choose the local addresses

Use `deploy/pi.example.json` as the starting settings file. Pick an address range for protected devices that differs from the existing router's range. For example, if the router uses `192.168.1...`, the example protected range is `192.168.50....`

The example Pi address on the protected side is `192.168.50.1`. Your friend should make each enrolled device keep a stable address. A stable-address assignment and adding the device to TrustEdge are separate tasks; doing one does not do the other.

The following settings names are included so your friend can find them in the file:

Setting in the file	Everyday meaning
`lan`	The protected-device connection, normally the USB Ethernet adapter.
`wan`	The connection toward the existing router, normally built-in Ethernet.
`lan_address`	The Pi's address on the protected side and the size of that local address range.
`dhcp_start` and `dhcp_end`	The first and last addresses the Pi can hand out automatically.
`reservations`	The stable addresses assigned to particular devices.
`dns_upstream`	The chosen service for looking up outside website names. An empty list means local names only.
`database`	Where real device information and learned behavior are saved on the Pi.
`masquerade`	The router-compatibility setting. Keep the example value unless your friend has arranged another return route.

Do not change the existing router's settings by guesswork. If its address range is unknown, establish that first.

Step 5 Generate and review the setup files

The project can generate the required Pi setup files from those settings. This command prepares files without changing the computer's network:

```
.venv/bin/python -m trustedge.hardware render \
  --config deploy/pi.example.json --output build/my-pi
```

Run it from the project folder after editing the settings, and use a fresh output folder. It refuses to overwrite an existing generated set.

The generated files describe the Pi's protected-side address, its local address and website-name services, its traffic-routing rules, its initial blocking rules, and how TrustEdge starts automatically. Generating them does not install or activate them.

Step 6 Install the generated files on the Pi

Your friend must place the reviewed files in the locations listed in the technical deployment guide, arrange the local address and name services, and set up automatic startup. Existing network settings must be checked for conflicts first.

The startup order matters: the initial blocking rules must be active before the Pi starts passing device traffic onward. Otherwise there could be an unprotected period while the Pi starts. The project provides the setup files for that order, but the real boot behavior must be checked on the Pi.

The live installation also needs an administrative access key. Your friend sets it once in the Pi's protected settings file. The automatic sign-in launcher is only for the local simulation; it does not remove authentication from the live Pi.

Step 7 Connect and add one real device

With the initial Pi setup in place, attach the router, Pi, adapter, and one test device as shown in the diagram. Confirm that the device receives its intended address and uses the Pi as its way out to the network.

Add that address and a clear name through the dashboard. Choose the closest device category and list the outside addresses it normally needs. Start with a test laptop rather than assuming every smart device will work with the initial limited-access rules.

An unregistered device is intentionally blocked from forwarding ordinary traffic. A registered device initially has limited access while learning. These are different from a broken cable, so check both the physical connection and the dashboard state when diagnosing a problem.

Step 8 Observe normal behavior

Use the device in its expected, legitimate ways and let learning progress. Do not deliberately generate suspicious traffic during initial learning. Check the device's learning count and model status in the dashboard.

After enough suitable observations, confirm that the first model is saved and that later normal periods are processed. The second model needs additional qualifying normal observations. Do not promise completion after a fixed number of wall-clock minutes for a quiet device.

Step 9 Test and record the result

Run the checklist in Section 10 before treating the setup as protective. Start with one device and expand only after routing, learning, restrictions, and restart behavior have been confirmed.

9 How to open the real Pi dashboard

The live dashboard is kept local to the Pi. This prevents accidentally placing its administration page on the open Internet. On a Pi with a desktop browser, it is available at `http://127.0.0.1:8080/` on that Pi.

If you type that address on your Mac without an arranged connection, it refers to your Mac, not the Pi. To use the Pi's dashboard from your computer, your friend can create a private remote connection called an SSH tunnel:

```
ssh -L 8080:127.0.0.1:8080 USERNAME@PI_ADDRESS
```

Replace `USERNAME` with the account created on the Pi and `PI_ADDRESS` with its reachable router-side address. After that connection is open, visit `http://127.0.0.1:8080/` on your computer and use the live Pi's administrative key.

If the local demonstration is already using port 8080, stop it first or ask your friend to use another local port. Keep the remote connection open while using the dashboard. The Pi can continue observing devices after the browser is closed, as long as its installed service is running.

Check that the dashboard says **LIVE GATEWAY**, shows the correct Pi connections, and is reporting real capture health. A **SIMULATION** label means the demonstration is still running. Do not judge the Pi setup from the demo's fictional devices or old example scores.

10 Checklist before calling the Pi setup successful

Record the date, the Pi operating-system version, and the person checking each item. Keep the results with the project handover.

Connections and normal use

- The router-side and protected-device-side cables and connection names are correct.
- The protected address range does not conflict with the router's range.
- The test device gets the intended stable address and uses the Pi for its network access.
- The test device has no alternate Wi-Fi or cable route around the Pi.
- Normal name lookup and address renewal work with real services, not just test fixtures.

Learning and dashboard

- The dashboard identifies live mode and shows the actual Pi configuration.
- Real capture is running and recent observations belong to the real enrolled device.
- Suitable normal observations advance learning; learned state survives a service restart.
- Device names, graphs, buttons, confirmations, keyboard controls, and the animated map work in the chosen browser.
- The map clearly distinguishes confirmed restrictions, pending changes, and illustrative animation.

Restriction and recovery

- A device not added to the project cannot forward ordinary traffic.
- CAUTIOUS access permits the expected listed destinations and blocks other ordinary destinations.

- Repeated suspicious test periods cause gradual, confirmed restrictions rather than an immediate quarantine from one scan.
- QUARANTINE blocks ordinary outgoing traffic, including a connection already open before the restriction.
- A deliberate reviewed release grants only CAUTIOUS access and records the reason.
- Restarting learning does not silently release quarantine or grant full access.
- Removing a device does not accidentally unblock its address.

Restart and reliability

- The two Linux-only automated checks run successfully on a suitable Linux/Pi environment.
- An actual Pi reboot preserves saved trust, learning, models, warning counts, and device restrictions.
- Traffic is denied until the startup protection is in place.
- A failed rule change is visibly reported and is not labeled successful.
- A stopped traffic collector makes the dashboard show that the gateway needs attention.
- Processor use, memory, storage, temperature, and traffic speed are acceptable with the intended number of devices.
- No TrustEdge telemetry uploads, remote model downloads, or monitoring emails occur.

The automated Linux checks use isolated software test networks. They are valuable but do not replace actual cable, device, reboot, and browser checks. Their address-renewal and name-service fixtures check access to the relevant ports, not the complete real services.

11 Common questions during setup

Why is a device blocked immediately after connection

First check whether it has been added to TrustEdge and whether its address matches the saved entry. Unknown devices are denied ordinary forwarding. Also check the cable, the protected-side address, and whether the device has an unintended alternative connection.

Why is learning taking longer than 50 minutes

Only suitable periods with enough packets count. A quiet printer or sensor may take longer. Repeated suspicious periods are not clean learning examples. The initial limited-access rules can also prevent a device from showing all of its normal behavior; your friend must review whether its required traffic is supported.

Why does the score differ from the access level

The project waits for repeated evidence, learning completion, and successful rule application. A score alone does not immediately change access. Read both the desired and confirmed columns and inspect recent activity.

Why does a quarantined device remain quarantined after a restart

Restrictions are saved deliberately. A restart is not an automatic release. Review the cause, then use the explicit release action if appropriate. Starting learning again is a separate operation.

Why does the map move when there is no measured traffic

The animation illustrates the network arrangement and policy paths. It is not a live packet counter. Use the traffic page for actual retained observations. The map pauses when the gateway is unhealthy or its connection becomes stale.

Does the project read messages or send them to a cloud service

The saved measurements describe traffic behavior rather than message contents. Packet payloads are not retained by this project. TrustEdge does not upload monitoring data or fetch replacement models. The watched devices may still contact their own services when their access rules allow it.

Can we assume every device will work without changing anything

No. Device addresses, connection names, required destinations, and needed traffic types must be checked. The current cautious policy is limited to particular web traffic. The initial hardware trial should establish what needs extending before expanding to other device types.

12 What to hand over to your friend

Provide this complete Phase 2 source project, this guide, and the related files listed below. Separately provide any trusted saved Phase 1 model files that you actually have. Do not include administrative secrets in a broadly shared copy.

File	When your friend should use it
`START_HERE.md`	First orientation and a summary of the current development boundary.
`docs/PI_DEPLOYMENT.md`	Exact installation locations, services, commands, and hardware checks.
`docs/DASHBOARD.md`	Launching the dashboard, understanding its pages, and live access.
`docs/PHASE1_ML.md`	Comparing Phase 1 behavior and importing separately saved trained models.
`docs/PHASE2_PLAN.md`	The component migration map and remaining development work.
`deploy/pi.example.json`	The editable starting settings for the real Pi.
`tests/`	The checks to run before and during integration.

Before the first hardware run, agree on the Pi operating system, the two actual connection names, the protected address range, one initial test device, and the person responsible for reviewing restrictions. After the run, record what passed, what failed, and what changes were needed.

The handover position: the software foundation and demonstration are implemented. Real-device integration is the next step, and the Pi connection is complete only when installation and the practical checks have passed.